SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Programme: **B.E / B.Tech**

Course Code: **CSA17**

Course Name: **ARTIFICIAL INTELLIGENCE**

Submission Date:6-11-2026

Date of Issue :4-11-2026 Max Marks: **50 Marks**

ASSIGNMENT – COURSE OUTCOME 2

Course Outcomes

On successful completion of the course, students will be able to:

CO	Course Outcome Statement
CO2	Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently.

Evidence Rule Common to All Assignments

Every implemented task must be supported by: **Source Code → Test Input → Expected Behaviour → Actual Output → Performance Metrics (table/graph over at least three input sizes or seeds) → Interpretation.** A screenshot showing only a final answer is not sufficient. Operation counts and task metrics are primary evidence; execution time is secondary. Wherever a task says *Predict*, the prediction must be written before the experiment is run. Programming language: Python 3 (Jupyter / Colab).

CO2: Informed search strategies, greedy best-first search, A* search, heuristic functions, local search algorithms and optimization problems, online search agents, constraint satisfaction problems, adversarial search, minimax algorithm, alpha-beta pruning and evaluation functions.

Assignment Title: Design, Analyse and Validate an Efficient Heuristic Search Strategy for Smart-City Ambulance Routing **(Total Marks – 50)**

Scenario — Smart-City Ambulance Routing

A smart-city emergency control room dispatches ambulances from the nearest hospital to accident sites. The city is modelled as a road graph of junctions with travel times on every road, and the GPS coordinates of every junction are known. A heuristic such as straight-line distance divided

by the maximum allowed speed can estimate the remaining travel time. Every second saved can save a life, but the routing server handles hundreds of requests per minute.
As the service is extended from one city to the whole district and then the state, the road graph may grow:
1,000 → 10,000 → 100,000 → 1,000,000+ junctions
You are required to investigate, design and compare informed (heuristic) search strategies by which the control room finds the fastest route from the hospital to the accident site as the road network grows.
Your solution must remain practical for both small and very large road networks. Any preprocessing (for example, building adjacency lists, computing heuristic values or a priority-queue structure) must be included in your efficiency analysis.
Design Challenge (Day 4): During a cyclone alert, several roads are flooded and closed without warning, and the ambulance discovers a closure only when it reaches the junction. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Design the Agent and the Heuristic Search Algorithms (5 Marks)
Before designing algorithms, define the agent, the problem and the heuristic:

- Specify the PEAS description of the ambulance-routing agent and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice
- Design at least two heuristic functions for travel time, prove which of them are admissible and consistent, and explain how each could be derived from a relaxed problem
- Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** informed search strategies (for example, greedy best-first search, A*, weighted A*, IDA* or RBFS). Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.
Your proposed approaches must include:

- At least one iterative approach (priority-queue based)
- At least one recursive approach (for example, IDA* or recursive best-first search)
- A clear explanation of the basic idea behind each approach and the evaluation function f(n) it uses
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required, such as adjacency-list construction, heuristic pre-computation or a closed set

You are expected to select the approaches independently. The appropriateness and diversity of the selected approaches are part of the assessment.
Evidence Expected: PEAS and environment classification, heuristic design with admissibility and consistency proofs, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

TASK 2 – Identify the Basic Operation (2 Marks)
For each proposed algorithm, identify the basic operation that predominantly determines its execution time.
Discuss:

- What the basic operation is (for example, node expansion, heuristic evaluation, priority-queue insertion or extraction)
- Where it occurs in the algorithm
- Why it is appropriate for efficiency analysis
- How the number of times it executes may change as the road network grows and as the heuristic becomes more or less accurate

Do not simply name an operation. Relate it to the behaviour of the algorithm.
Evidence Expected: Correct identification and justification of the basic operation for each algorithm.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)
Derive the time and space complexity for each proposed approach using mathematical analysis.
Discuss:

- The input size (branching factor b, solution depth d, effective branching factor b*, optimal cost C*, minimum step cost ε, heuristic error h* − h)
- The number of times the basic operation executes, derived from the nodes whose $f(n) < C^*$ at each level
- The mathematical expression for time and for memory (open and closed lists for A*, recursion stack for IDA*/RBFS)
- Any preprocessing cost involved (heuristic computation for every junction, priority-queue operations of O(log n))
- The overall cost when preprocessing and processing are considered together

You must show the mathematical reasoning. Merely stating a Big-O result will not receive full credit. **Evidence Expected:** A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)
Analyse each algorithm under different input conditions.
Discuss, wherever applicable:

- What input condition produces the best case (for example, a perfect heuristic $h = h^*$, so only nodes on the optimal path are expanded)
- What condition produces the average case (a heuristic with small constant or relative error)
- What condition produces the worst case (for example, $h = 0$ so A* behaves like uniform-cost search, a misleading heuristic for greedy search, many distinct f-values for IDA*)
- How the number of nodes expanded changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

Evidence Expected: A justified best-, average- and worst-case analysis supported by mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis (3 Marks)
Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ.
Discuss:

- How each algorithm behaves as the input becomes very large
- How the growth rates of the approaches differ in time and in memory
- What the growth means when the road network grows from 1,000 to 1,000,000+ junctions, for example, whether the memory of A* or the re-expansions of IDA* become the limiting factor first

Interpret the behaviour in the context of ambulance routing rather than merely stating the notation.
Evidence Expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

TASK 6 – Formulate the Recurrence Relation (3 Marks)
For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm. Identify clearly:

- The size of the original problem (the f-cost limit or the remaining depth within the current contour)
- The size of the recursive subproblem
- The number of recursive calls (related to the branching factor and the nodes with $f(n) \leq$ limit)
- The work performed outside the recursive calls
- The base case
- How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook.
Evidence Expected: A correct recurrence with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation (4 Marks)
Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).
Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the recursive algorithm as the input size increases

- How the result extends to IDA*, where the contour search is repeated for increasing f-limits, and how the number of distinct f-values affects the total cost

Show the analytical process, not only the final result.
Evidence Expected: A complete solution with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms (4 Marks)
Implement all proposed algorithms in Python and test them experimentally.
Use generated road networks (with junction coordinates and travel times) of approximately: 1,000 → 10,000 → 100,000 → 1,000,000 junctions Ensure that:

- All algorithms return valid routes, and A* returns optimal routes with an admissible heuristic
- The same road networks and hospital/accident pairs are used for comparison
- Testing conditions are consistent and the random seeds are reported
- Test cases include a reachable site, an unreachable site and a site at the hospital itself, and one run with an inadmissible heuristic
- The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.
Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

TASK 9 – Record Experimental Performance (3 Marks)
Collect and present performance data for each algorithm across the different input sizes.
Record, with code and outputs as proof:

- Nodes expanded, heuristic evaluations and effective branching factor b*
- Maximum size of the open list or recursion depth (memory proxy)
- Route cost, and whether the route is optimal (compare with A* or Dijkstra)
- Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.
Evidence Expected: Systematically collected experimental data with clear tables and graphs.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)
Compare your theoretical analysis with the experimental results.
Discuss:

- Whether the observed number of nodes expanded and memory use agree with the theoretical complexity and with the heuristic quality
- Which algorithm scales better as the road network grows

- Any differences between theory and observation, and the possible reasons (graph structure, heuristic error, tie-breaking, re-expansions, language overhead)

Interpret the results rather than simply reproducing tables or graphs.
Evidence Expected: A reasoned comparison between theoretical and observed performance.

TASK 11 – Recommend and Defend the Final Design (4 Marks)
Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for smart-city ambulance routing.
Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives
- Its theoretical efficiency and experimentally observed performance
- Any preprocessing cost, trade-offs or limitations
- Whether it remains practical as the network grows to state level

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest BigO complexity is not sufficient.
Evidence Expected: A technically justified recommendation supported by analysis and experiment.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)
Changed Requirement
When a cyclone alert is issued, a fraction of roads are flooded and closed. The routing agent does not know which roads are closed until the ambulance reaches the junction where the road starts, and travel times on open roads may change. It must still reach the accident site, whichever roads turn out to be closed.
Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis – 2 Marks

- What has changed from the original problem and which original assumptions (known map, static, deterministic costs) are affected?
- Why does the agent now need to act and plan at the same time?

B. Evaluation of the Original Design – 2 Marks

- Can your original algorithm still be applied? What happens to the optimality guarantee and to the heuristic's admissibility?
- How many re-plans could be needed when *c* roads on the route are closed, and is the original design acceptable?

C. Adaptation or Redesign – 2 Marks

- Describe the revised strategy (for example, re-planning A* after each discovery, or an online search agent such as LRTA* that learns heuristic values), what has changed and the effect on complexity and on the competitive ratio
- If you retain the original solution, give sufficient technical justification

D. Evidence-Based Defence – 2 Marks

- Defend your decision using mathematical analysis and a small experiment (route cost, nodes expanded and number of re-plans for 0 %, 5 % and 10 % closed roads, with re-planning A* and with an online search agent), with code and output as proof

RUBRICS FOR EVALUATION

Task / Criteria	Marks	Level 4 – Exemplary (90–100%)	Level 3 – Proficient (70–89%)	Level 2 – Developing (40–69%)	Level 1 – Beginning (0–39%)
Task 1 – Agent, heuristic and search strategies	5	Scenario-specific PEAS; all six properties justified; two heuristics with admissibility/consistency proofs; complete formulation; three distinct informed strategies (iterative and recursive) with pseudocode and justification.	PEAS, heuristics and formulation complete with minor gaps; three strategies with pseudocode.	Generic PEAS; heuristic not justified; strategies not distinct or pseudocode missing.	Agent, heuristic or formulation missing; fewer than two strategies.
Task 2 – Basic operation	2	Correct basic operation for every algorithm, located in the code and related to algorithm behaviour and growth.	Correct operations with brief justification.	Operations named without justification.	Incorrect or missing.
Task 3 – Complexity derivation	6	Node counts derived from f-contours and effective branching factor for each approach; open/closedlist and stack memory; preprocessing cost	Derivations mostly complete; minor slips or preprocessing omitted.	Big-O stated with little derivation.	No derivation.

		included; correct combined cost.			
Task 4 – Best, average and worst case	4	Clear input conditions for each case and algorithm, with operation-count reasoning.	Cases correct; reasoning partly given.	Table of values without conditions.	Missing or incorrect.
Task 5 – Asymptotic analysis	3	Correct O/Ω/Θ for time and memory; growth from 1,000 to 1,000,000+ junctions interpreted for ambulance routing.	Correct notation; limited interpretation.	Notation partly wrong; no interpretation.	Missing.
Task 6 – Recurrence formulation	3	Recurrence derived from own IDA*/RBFS algorithm with f-limit, subproblems, calls, outside work and base case identified.	Correct recurrence; one component unclear.	Recurrence copied or components missing.	Missing or wrong.
Task 7 – Recurrence solution	4	Method justified; all steps shown; correct result interpreted and extended to repeated IDA* contours.	Correct solution with minor gaps in steps or interpretation.	Final answer only, or method misapplied.	Missing or wrong.
Task 8 – Implementation and testing	4	All algorithms working on all four network sizes; seeds reported; reachable, unreachable, start = goal and inadmissible-heuristic cases tested; code matches analysis.	Working code; one test case type or size missing.	Partial implementation or untested.	No working code.
Task 9 – Performance recording	3	Nodes expanded, heuristic evaluations, b*, memory, cost, optimality and	Most metrics recorded; graphs basic.	Few metrics; no graphs.	No data.

		averaged time recorded for all sizes, with clear tables and log-scale graphs.			
Task 10 – Theory vs experiment	4	Observed growth compared with theory for every algorithm; differences explained with sound reasons.	Comparison made; reasons partly explained.	Tables repeated with little interpretation.	Missing.
Task 11 – Recommendation	4	Recommendation backed by own analysis and evidence; tradeoffs, preprocessing and scalability discussed.	Reasonable recommendation; evidence partly used.	Recommendation based only on Big-O.	Missing or unjustified.
Task 12 – Day-4 re-evaluation	8	Impact analysis, evaluation, adaptation (re-planning / online search) and defence all complete, with experiment for 0 %, 5 % and 10 % closed roads.	Three of four parts strong; experiment partial.	Generic discussion; no experiment.	Missing.

Total: 50 marks. Each task is marked out of the marks shown by choosing the level that best matches the evidence submitted and awarding the matching percentage of that task's marks (round to the nearest 0.5). A task without source code, test input and actual output as proof (Evidence Rule) cannot score above Level 2.

CODE OF CONDUCT

I A.Khaja Hussain (192572118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assignment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A.Khaja Hussain

TASK 1 – Design the Agent and the Heuristic Search Algorithms

1. PEAS Description

PEAS stands for Performance Measure, Environment, Actuators and Sensors.

Component	Description
Performance Measure	Reach the accident location in minimum time, save lives and reduce travel distance.
Environment	City roads, hospitals, junctions, traffic conditions and accident locations.
Actuators	Ambulance engine, steering, brakes, GPS navigation and route selection system.
Sensors	GPS, traffic sensors, road cameras and location-tracking devices.

2. Environment Classification

1. Partially Observable: The ambulance may not know the complete road and traffic conditions.
2. Deterministic: Normally, travelling through a road has a known travel cost, but unexpected events can change it.
3. Sequential: Each road selected affects the next movement and the final destination.
4. Dynamic: Traffic conditions and road availability can change continuously.
5. Continuous: The ambulance moves through physical roads, and travel time and distance are continuous values.
6. Multi-Agent: Other vehicles, ambulances and traffic-control systems also affect the environment.

3. Heuristic Functions

A heuristic function estimates the remaining cost from the current junction to the accident location.

Heuristic 1: Straight-Line Distance

$$h_1(n) = \frac{\text{Euclidean Distance}(n, Goal)}{V_{\max}}$$

- It calculates the straight-line distance between the current junction and the accident location.
- It divides the distance by the maximum allowed speed.
- It estimates the minimum possible remaining travel time.
- It is admissible because the actual road route cannot be faster than travelling in a straight line at maximum speed.
- It is consistent because the Euclidean distance satisfies the triangle inequality.

Heuristic 2: Landmark-Based Heuristic

$$h_2(n) = \max_L |d(L, Goal) - d(L, n)|$$

Where:

- L represents a selected landmark.
- d(L, Goal) is the shortest travel time from the landmark to the goal.
- d(L, n) is the shortest travel time from the landmark to the current junction.
- It uses precomputed distances from selected landmarks.
- It estimates the remaining travel time more accurately in many road networks.
- It is admissible because it never overestimates the actual shortest travel cost.
- It is consistent for an undirected road graph with non-negative edge costs.
- For directed roads, suitable forward and reverse landmark distances must be used.

4. Problem Formulation

Element	Description
States	Different road junctions in the city.
Initial State	Hospital location where the ambulance starts.
Actions	Move from one junction to another connected junction.
Transition Model	The ambulance moves along an available road to the next junction.
Goal Test	Checks whether the ambulance has reached the accident location.
Path Cost	Total travel time required to reach the accident location.

5. Three Informed Search Algorithms

A. Greedy Best-First Search

- Greedy Best-First Search selects the junction that appears closest to the destination.
- It uses only the heuristic value to select the next junction.
- It tries to reach the goal quickly.
- It does not always find the shortest or fastest route.

Evaluation function:

$$[f(n) = h(n)]$$

Pseudocode:

1. Insert the starting node into priority queue.
2. Select the node with the lowest heuristic value.
3. Check whether it is the goal.
4. If it is the goal, return the path.

5. Otherwise, expand its neighbouring nodes.
6. Insert unvisited neighbours into the queue.
7. Repeat until the goal is found.

B. A Search Algorithm*

- A* combines the actual cost and estimated remaining cost.
- It selects the junction with the lowest total estimated cost.
- It uses both the distance already travelled and the remaining estimated travel time.
- With an admissible heuristic, it finds an optimal route under the standard graph-search conditions.

Evaluation function:

$$[\, f(n) = g(n) + h(n)]$$

Where:

- g(n) = Actual travel time from the hospital.
- h(n) = Estimated remaining travel time.

Pseudocode:

1. Insert the starting node into the priority queue.
2. Calculate g(n) + h(n).
3. Select the node with the lowest f(n).
4. Check whether it is the goal.
5. If it is the goal, return the path.
6. Expand its neighbouring nodes.
7. Update their costs and insert them into the queue.
8. Repeat until the goal is found.

C. Iterative Deepening A (IDA)**

- IDA* combines A* search with depth-first search.
- It uses a cost limit to control the search.
- It explores nodes whose estimated total cost is within the current limit.
- If the goal is not found, it increases the limit and searches again.
- It requires less memory compared with A*.

Evaluation function:

$$[\, f(n) = g(n) + h(n)]$$

Pseudocode:

1. Set the initial limit as h(start).
2. Start depth-first search.
3. Calculate f(n) = g(n) + h(n).
4. If f(n) exceeds the limit, return the exceeded value.
5. If the goal is reached, return the path.
6. Explore the neighbouring nodes.
7. Increase the limit using the minimum exceeded value.

8. Repeat until the goal is found.

6. Preprocessing Requirements

- Convert the road network into an adjacency list.
- Store travel times between connected junctions.
- Store GPS coordinates of all junctions.
- Calculate heuristic values.
- Prepare a priority queue for Greedy and A*.
- Prepare a recursion or search stack for IDA*.

7. Justification for Selecting the Algorithms

- Greedy Best-First Search is selected because it can quickly identify promising routes.
- A* is selected because it provides optimal routes with an admissible heuristic.
- IDA* is selected because it uses less memory and is useful for large search spaces.

TASK 2 – Identify the Basic Operation

The basic operation is the main operation that determines the execution time of an algorithm.

1. Greedy Best-First Search

- The basic operation is node expansion.
- It occurs when the algorithm selects a junction and checks its neighbouring junctions.
- It also calculates heuristic values for the neighbouring nodes.
- The number of operations increases when the road network becomes larger.
- A good heuristic reduces unnecessary node expansions.

2. A* Search

- The basic operation is calculating the total estimated cost and processing the priority queue.
- It occurs when the algorithm calculates $(f(n) = g(n) + h(n))$.
- It selects the junction with the lowest estimated cost.
- The number of operations increases as more junctions and roads are explored.
- A better heuristic reduces the number of expanded nodes.

3. IDA* Search

- The basic operation is recursive node expansion.
- It occurs when the algorithm explores neighbouring junctions using depth-first search.
- It repeatedly calculates the estimated cost.

- The number of operations increases when the search limit is increased.
- A poor heuristic causes repeated exploration and increases execution time.

TASK 3 – Derive the Time and Space Complexity Mathematically

1. Important Parameters

Symbol	Meaning
b	Branching factor (number of neighbouring nodes).
d	Depth of the optimal solution.
b*	Effective branching factor.
C*	Optimal path cost.
ε	Minimum cost of one step.
h*	Actual remaining cost.
h	Estimated remaining cost.
n	Number of junctions.

2. Greedy Best-First Search

Time Complexity:

At each level, the algorithm may explore multiple nodes.

$$[\,1 + b + b^2 + \cdots + b^d]$$

Using the geometric series:

$$[\,T(n) = O(b^d)]$$

If all junctions and roads are explored, the graph-search bound is approximately:

$$[\,T(n) = O\big((V + E)\backslash log\ V\big)]$$

for a binary-heap priority queue.

Space Complexity:

Greedy search stores the frontier and visited nodes.

$$[\,S(n) = O(V)]$$

3. A* Search

A* expands nodes whose estimated total cost is less than or equal to the optimal cost.

$$[\,N = \sum_{\{i=0\}_i^{\{d\}N}]}$$

Where ($N_{i)}$ represents the number of nodes expanded at level i.

In the worst case:

$$[\,T(n) = O(b^d)]$$

With a priority queue, each insertion or removal takes approximately:

$$[\,O(\backslash log\ V)]$$

Therefore, graph-search processing can be bounded by:

$$[\,T(n) = O\big((V + E)\backslash log\ V\big)]$$

Space Complexity:

A* stores open and closed lists.

$$[\,S(n) = O(V)]$$

4. IDA* Search

IDA* uses repeated depth-first searches.

The number of nodes explored in one search contour is approximately:

$$[\,1 + b + b^2 + \cdots + b^d]$$

Therefore:

$$[\,T(n) = O(b^d)]$$

Repeated contours may increase the total number of node expansions.

Space Complexity:

IDA* stores the current search path and recursion stack.

$$[\,S(n) = O(d)]$$

5. Preprocessing Cost

- Adjacency-list construction requires $(O(V + E))$ time.
- Storing coordinates requires $(O(V))$ space.
- Calculating straight-line heuristic values for every junction requires $(O(V))$ time for a fixed goal.
- Priority-queue operations require $(\mathrm{O}(\backslash \log \mathrm{V}))$ time each.

Overall Processing Cost:

$$[\,T_{\{\backslash text\{total\}\}} = T_{\{\backslash text\{preprocessing\}\}} + T_{\{\backslash text\{search\}\}}]$$

Preprocessing is important because it affects the total time required to generate an ambulance route.

TASK 4 – Best, Average and Worst-Case Behaviour

1. Greedy Best-First Search

Best Case:

- The heuristic correctly identifies the direction of the accident location.
- The ambulance follows the correct route.
- Very few junctions are expanded.

Average Case:

- The heuristic provides a reasonably accurate estimate.
- Some unnecessary junctions are explored.
- Execution time remains moderate.

  Worst Case:
- The heuristic gives misleading estimates.
- The algorithm explores many unnecessary junctions.
- It may select a longer route.

2. A* Search

Best Case:
- The heuristic is perfect, meaning \(h(n)=h^*(n)\).
- Only the nodes necessary for the optimal route are expanded, apart from possible equal-cost ties.

  Average Case:
- The heuristic is close to the actual remaining cost.
- A moderate number of nodes are expanded.
- The algorithm finds an optimal route efficiently.

  Worst Case:
- The heuristic is zero.
- A* behaves like Uniform Cost Search.
- It may explore a very large number of junctions.

3. IDA* Search

Best Case:
- The heuristic is accurate.
- The correct route is found with very few search iterations.
- Memory usage remains low.

  Average Case:
- The heuristic provides a reasonable estimate.
- A few additional search contours may be required.
- Some nodes are explored repeatedly.

  Worst Case:
- The heuristic is poor.
- Many search contours are required.
- The same nodes may be expanded multiple times, increasing execution time.

TASK 5 – Perform Asymptotic Analysis

Asymptotic analysis explains how the performance of an algorithm changes when the input size increases.

1. Greedy Best-First Search

- Best case: $(\Omega(\mathrm{d}))$, when the correct path is selected directly.
- Worst case: $(\mathrm{O}(\mathrm{V}+\mathrm{E}))$ for graph search with a suitable priority queue, including graph exploration.
- Space complexity: $(O(V))$.
- It may expand many unnecessary junctions when the heuristic is misleading.

2. A* Search

- Best case: $(\backslash \mathrm{Omega}(\mathrm{d}))$, when a perfect heuristic guides the search along the optimal path.
- Worst-case graph-search time: $(\mathrm{O}\big((\mathrm{V}+\mathrm{E})\backslash \log \mathrm{V}\big))$ with a binary heap and appropriate implementation.
- Space complexity: $(O(V))$.
- It may require large memory because it stores explored and unexplored junctions.

3. IDA* Search

- Best-case time: approximately $(\backslash Omega(d))$ when the search follows the correct path.
- Worst-case time: $(\mathrm{O}\big(\mathrm{b}^{\mathrm{d}}\big))$ per contour, with additional cost from repeated contours.
- Space complexity: $(O(d))$.
- It saves memory but may take more time because it explores nodes repeatedly.

Comparison

Algorithm	TimeComplexity	SpaceComplexity
Greedy Best − First	$(\mathrm{O}\big((\mathrm{V}+\mathrm{E})\backslash \log \mathrm{V}\big))$	$(\mathrm{O}(\mathrm{V}))$
A ∗	$(\mathrm{O}\big((\mathrm{V}+\mathrm{E})\backslash \log \mathrm{V}\big))$	$(\mathrm{O}(\mathrm{V}))$
IDA ∗	$(\mathrm{O}\big(\mathrm{b}^{\mathrm{d}}\big))$ per contour	$(\mathrm{O}(\mathrm{d}))$

Interpretation for Smart-City Ambulance Routing

- When the road network increases from 1,000 to 1,000,000 junctions, the number of explored junctions also increases.
- Greedy search may find routes quickly but cannot guarantee the optimal route.
- A* provides optimal routes but requires more memory.

- IDA* requires less memory but may repeat the same search.
- For emergency routing, A* provides a good balance between route quality and search efficiency when sufficient memory is available.

TASK 6 – Formulate the Recurrence Relation

For the recursive approach, we use the IDA* algorithm.

1. Size of the Problem

The problem size is represented by the remaining cost limit available for searching.

Let:

- $(T(k))$ = Number of operations for the current search.
- (b) = Branching factor.
- (k) = Remaining search depth or cost budget.

2. Recursive Subproblem

Each junction can generate up to b neighbouring junctions.

The algorithm recursively explores the neighbours whose estimated total cost does not exceed the current limit.

3. Recurrence Relation

For a simplified worst-case analysis:

$$[\, T(k) = bT(k-1) + 1 \,]$$

Where:

- $(bT(k-1))$ represents the recursive calls.
- 1 represents the work performed at the current node.

4. Base Case

$$[\, T(0) = 1 \,]$$

When the search reaches the limit or there are no further valid nodes, recursion stops.

5. Explanation

- IDA* starts from the initial junction.
- It calculates the estimated cost of each neighbouring junction.
- It recursively explores valid neighbours.
- Each recursive call reduces the remaining search depth.
- The process continues until the goal is found or the search limit is exceeded.

The recurrence represents the worst-case branching behaviour. In actual IDA*, the number of recursive calls depends on the f-cost limit and the heuristic values.

TASK 7 – Solve the Recurrence Relation

The recurrence relation is:

$$[\ T(k) = bT(k-1) + 1\ ]$$

With the base condition:

$$[\ T(0) = 1\ ]$$

1. Iteration Method

Substitute the recurrence repeatedly.

First iteration:

$$[\ T(k) = bT(k-1) + 1\ ]$$

Second iteration:

$$[\ T(k) = b[bT(k-2) + 1] + 1\ ]$$

$$[\ T(k) = b^{2T(k-2)} + b + 1\ ]$$

Third iteration:

$$[\ T(k) = b^{3T(k-3)} + b^2 + b + 1\ ]$$

After k iterations:

$$[\ T(k) = b^{kT(0)} + b^{\{k-1\}} + \cdots + b + 1\ ]$$

Since $(T(0) = 1)$:

$$[\ T(k) = 1 + b + b^2 + \cdots + b^k]$$

Using the geometric series formula:

$$[\ T(k) = \backslash frac\{b^{\{k+1\}} - 1\}\{b - 1\}]$$

Therefore:

$$[\ \backslash boxed\{T(k) = O(b^k)\}]$$

2. Interpretation

- The execution time increases exponentially with the search depth in the worst case.
- When the branching factor increases, the number of recursive calls also increases.
- IDA* reduces memory usage but may repeat node expansions.
- Multiple search contours increase the total execution time.
- The number of distinct f-values determines how many different search limits may be needed.

3. Application to IDA*

IDA* begins with an initial f-cost limit and increases it whenever the goal is not found.

For example:

- First search: limit = 10.
- Second search: limit = 15.
- Third search: limit = 20.
- Fourth search: limit = 25.

The algorithm continues until it reaches the goal. Each repeated search may explore some previously visited junctions.

TASK 8 – Implement and Test the Algorithms

This task mainly requires practical implementation, but the following is the theory and testing procedure.

1. Implementation Description

Three algorithms are implemented using Python 3:

1. Greedy Best-First Search.
2. A* Search.
3. Iterative Deepening A*.

All algorithms use the same generated road network to ensure fair comparison.

2. Test Input Sizes

The algorithms are tested using different road network sizes.

Test Case	Number of Junctions
Test 1	1,000
Test 2	10,000
Test 3	100,000
Test 4	1,000,000

3. Test Conditions

- Reachable destination: The ambulance can reach the accident location.
- Unreachable destination: No available road connects the starting point to the destination.
- Start equals goal: The hospital and accident location are the same.
- Inadmissible heuristic: A heuristic that overestimates the actual remaining travel time is tested.

4. Expected Behaviour

- Greedy search should find a route quickly but may not find the optimal route.
- A* should find an optimal route when an admissible heuristic and suitable graph-search implementation are used.
- IDA* should find an optimal route under the same admissibility assumptions but may take additional time.
- All algorithms should correctly identify an unreachable destination.
- When the starting point is the goal, the algorithm should immediately return a zero-cost route.

TASK 9 – Record Experimental Performance

This task focuses on measuring the performance of the three algorithms.

1. Performance Parameters

1. Nodes Expanded: Number of junctions explored by the algorithm.
2. Heuristic Evaluations: Number of times the heuristic function is calculated.
3. Effective Branching Factor: Average number of useful successor nodes generated during the search.
4. Memory Usage: Maximum number of nodes stored in the open list or recursion stack.
5. Route Cost: Total travel time from the hospital to the accident location.
6. Execution Time: Total time taken to find the route.

2. Performance Measurement Table

The following table can be used to record experimental results.

Junctions	Algorithm	Nodes Expanded	Execution Time	Memory Used
1,000	Greedy	To be measured	To be measured	To be measured
1,000	A*	To be measured	To be measured	To be measured
1,000	IDA*	To be measured	To be measured	To be measured
10,000	Greedy	To be measured	To be measured	To be measured
10,000	A*	To be measured	To be measured	To be measured
10,000	IDA*	To be measured	To be measured	To be measured
100,000	All three	To be measured	To be measured	To be measured
1,000,000	All three	To be measured	To be measured	To be measured

3. Performance Analysis

- The number of expanded nodes indicates how efficiently an algorithm searches the road network.
- Heuristic evaluations show the amount of estimation work performed.
- Memory usage indicates how much storage is required.

- Route cost helps determine whether the algorithm finds the fastest route.
- Execution time shows how quickly the ambulance routing system produces a result.
- At least five runs should be performed for each test condition, and the average execution time should be calculated.

4. Graphical Representation

The results should be presented using:

- Bar graph comparing nodes expanded.
- Line graph comparing execution time.
- Bar graph comparing memory usage.
- Graph comparing route costs.

A logarithmic scale can be used when the values differ greatly.

TASK 10 – Compare Theory with Experimental Behaviour

The theoretical analysis explains the expected performance of the algorithms, while experimental analysis measures their actual performance.

1. Greedy Best-First Search

- It is expected to expand fewer nodes when the heuristic is accurate.
- It may take a longer route because it does not consider the actual cost already travelled.
- Its performance depends heavily on heuristic accuracy.

2. A* Search

- It is expected to find the optimal route with an admissible heuristic.
- Its number of expanded nodes decreases when the heuristic becomes more accurate.
- Its memory usage can increase because it stores many explored junctions.
- Experimental results may show additional time due to priority-queue operations.

3. IDA* Search

- It uses less memory than A*.
- It may expand the same junctions repeatedly.
- Its execution time may increase because of repeated search contours.
- Its memory requirement is generally lower because it mainly stores the current path.

4. **Theory and Experiment Comparison**

Factor	Theoretical Analysis	Experimental Behaviour
Search time	Based on mathematical complexity	Measured execution time
Node expansion	Based on branching factor and heuristic	Actual expanded nodes
Memory	Based on open and closed lists or recursion stack	Actual maximum storage
Route cost	Based on optimal path calculations	Actual route travel time
Heuristic quality	Based on estimation accuracy	Observed search performance

5. Interpretation

- Theoretical and experimental results may not be exactly the same.
- Road network structure can affect the number of expanded nodes.
- A better heuristic usually reduces unnecessary exploration.
- Programming language overhead and priority-queue operations can affect execution time.
- A* is expected to provide the best balance when optimality is important and sufficient memory is available.

TASK 11 – Recommend and Defend the Final Design

1. Recommended Algorithm

I recommend the A Search Algorithm* for smart-city ambulance routing.

2. Reasons for Selecting A*

- A* finds the shortest travel-time route when an admissible heuristic is used.
- It considers both the actual travel time and the estimated remaining travel time.
- It avoids exploring many unnecessary junctions when the heuristic is accurate.
- It provides better route quality than Greedy Best-First Search.
- It is suitable for finding emergency routes in large road networks when sufficient memory is available.

3. Comparison with Other Algorithms

Factor	Greedy	A*	IDA*
Optimal route	Not guaranteed	Yes, under suitable conditions	Yes, under suitable conditions
Memory usage	High in worst case	High	Low
Search time	Can be fast	Depends on heuristic	Can be high
Repeated exploration	Possible	Usually limited	High
Route quality	May be poor	Optimal	Optimal

4. Limitations

- A* requires more memory for large road networks.
- Its performance depends on the quality of the heuristic.
- Preprocessing requires additional time and storage.
- Unexpected road closures require the algorithm to calculate a new route.

5. Final Conclusion

A* Search is the most suitable choice for the smart-city ambulance routing system because it finds an optimal route using actual and estimated travel costs. It helps reduce ambulance travel time and supports efficient emergency transportation. For very large road networks, memory optimization and preprocessing techniques can improve its performance.

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

A. Impact Analysis

1. Changes in the Original Problem

In the original problem, all roads are assumed to be available and their travel times are known. During a cyclone, some roads may become flooded or closed unexpectedly.

The following changes occur:

1. Some roads become unavailable.
2. Travel times on open roads may increase.
3. The ambulance may discover road closures only after reaching a junction.
4. The original road network information may become outdated.

5. The ambulance must find an alternative route.

2. Why Does the Agent Need to Act and Plan at the Same Time?

- The agent cannot know all future road conditions.
- It must move towards the accident location while collecting new road information.
- It must identify road closures using sensors and observations.
- It must update its route whenever new information becomes available.
- Therefore, the ambulance routing system needs an online decision-making approach.

B. Evaluation of the Original Design

1. Can the Original A* Algorithm Still Be Used?

Yes, the original A* algorithm can still be used, but it must be modified to handle changing road conditions.

- A* can calculate a new route when a road is closed.
- The previously calculated route may no longer be valid.
- The algorithm must update the road network.
- It must search for an alternative route.
- Its original optimality guarantee applies to the graph and costs used in that particular search, not necessarily to future unknown conditions.

2. Effect on Heuristic Admissibility

- A heuristic based on straight-line distance divided by maximum possible speed remains admissible if that speed is a valid upper bound.
- If road closures occur, the actual route may become longer.
- Therefore, the straight-line estimate will not overestimate the actual remaining travel time.
- However, a landmark heuristic based on old travel times must be checked when travel times can decrease.
- If costs only increase or roads close, old shortest-distance estimates remain lower bounds for the remaining available paths.

3. Number of Re-plans

Let c represent the number of closed roads discovered on the selected route.

The maximum number of searches using simple complete re-planning is:

$$[\boxed{\text{Total Searches}} = c + 1]$$

For example:

- If 0 roads are closed, 1 search is required.
- If 2 roads are closed, up to 3 searches may be required.
- If 5 roads are closed, up to 6 searches may be required.

Repeated complete searches may increase processing time.

C. Adaptation or Redesign

1. Proposed Revised Strategy

I recommend modifying the original A* algorithm using Dynamic Re-planning A*.
The system will calculate a new route whenever it discovers a road closure or a change in travel time.

2. Working Process

1. The ambulance starts from the nearest hospital.
2. A* calculates the fastest available route.
3. The ambulance starts travelling towards the accident location.
4. Sensors and GPS monitor road conditions.
5. If a road is closed, the system updates the road network.
6. A* calculates a new alternative route.
7. The ambulance follows the updated route.
8. This process continues until the accident location is reached.



3. Changes in Complexity

- Every re-planning operation requires additional search time.
- If c road closures are discovered, up to c+1 complete searches may be required.
- Memory usage remains similar to the original A* implementation for each search.
- The system needs additional processing to update road information.
- Re-planning helps the ambulance avoid closed roads and reach the destination.

4. Competitive Ratio

The competitive ratio compares the cost of the route followed by an online agent with the optimal route cost.

$$[CR = \frac{\text{Online Route Cost}}{\text{Optimal Route Cost}}]$$

A competitive ratio of 1 indicates an optimal route.

For dynamic road conditions, the ratio depends on how much information is available and when road closures are discovered. It cannot be guaranteed to equal 1 when future road conditions are unknown.

D. Evidence-Based Defence

1. Experimental Conditions

The revised system should be tested under three different road-closure conditions.

Test Case	Closed Roads	Expected Behaviour
Test 1	0%	Original route is followed.
Test 2	5%	Alternative route is calculated when required.
Test 3	10%	More re-planning may be required.

2. Performance Parameters

The following parameters should be measured:

- Total route cost.
- Number of nodes expanded.
- Number of re-planning operations.
- Total execution time.
- Memory usage.

3. Expected Results

- When 0% of roads are closed, the original A* route should be available.
- When 5% of roads are closed, the system may need to calculate alternative routes.
- When 10% of roads are closed, more route changes may be required.
- Dynamic re-planning helps the ambulance avoid unavailable roads.
- The system continues searching until it finds an available route or determines that no route exists.

These are expected behaviours, not measured experimental results. Actual values must be obtained by running the experiment.

4. Final Defence

I recommend modifying the original A* algorithm with dynamic re-planning because it can respond to unexpected road closures and changing traffic conditions. It updates the road network and calculates alternative routes whenever necessary. This approach improves the

reliability of ambulance routing during cyclone emergencies and helps the ambulance reach the accident location safely.